Lecture 40 Practice Problems: Deep Learning (Part-5)

CIFAR-10 CNN, NLP Foundations, Word Embeddings, RNN & LSTM

This problem set tests your conceptual, mathematical, and implementation mastery across computer vision pipelines, landmark convolutional neural networks, classical NLP vectorization, distributed word embeddings, recurrent neural networks, BPTT dynamics, and gated memory architectures (LSTM).

Problem 1: CIFAR-10 Normalization Transformation Mechanics

In PyTorch vision pipelines, CIFAR-10 images are transformed using:

```
transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
```

1. Given that `transforms.ToTensor()` scales raw 8-bit unsigned integer pixel values $p \in [0, 255]$ into floating-point numbers $x \in [0.0, 1.0]$, mathematically derive the range of output values produced by `transforms.Normalize`.
2. Why is zero-centered normalization strictly preferred over leaving inputs in the range $[0.0, 1.0]$ when training deep neural networks with gradient descent?

Problem 2: Deep CNN Feature Map Spatial Shape Calculation

Consider the 3-block convolutional neural network architecture implemented for CIFAR-10:

- **Input:** Tensor of shape `(Batch, 3, 32, 32)`
- **Block 1:** `Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)` followed by `ReLU()` and `MaxPool2d(kernel_size=2, stride=2)`
- **Block 2:** `Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)` followed by `ReLU()` and `MaxPool2d(kernel_size=2, stride=2)`
- **Block 3:** `Conv2d(in_channels=64, out_channels=128, kernel_size=3, padding=1)` followed by `ReLU()` and `MaxPool2d(kernel_size=2, stride=2)`

Compute the spatial height, width, and number of feature channels after each operation, and determine the exact number of nodes produced after flattening the output of Block 3.

Problem 3: Exact Parameter Count Analysis of ConvNet

For the network described in Problem 2, followed by:

- `Linear(in_features=2048, out_features=256)`
- `ReLU()`
- `Linear(in_features=256, out_features=10)`

Calculate the exact number of learnable parameters (weights and biases) for:

1. `Conv2d(3, 32, kernel_size=3, padding=1)`
2. `Conv2d(32, 64, kernel_size=3, padding=1)`
3. `Conv2d(64, 128, kernel_size=3, padding=1)`
4. `Linear(2048, 256)`
5. `Linear(256, 10)`
6. The total parameter count of the entire model.

Problem 4: Model Evaluation Mode & No-Grad Context

In PyTorch, during the test phase, practitioners wrap evaluation code with:

```
model.eval()
with torch.no_grad():
    # evaluation loop
```

1. What specific internal layer behaviors are modified by calling `model.eval()`, specifically regarding Dropout and Batch Normalization?
2. What is the explicit technical purpose of `with torch.no_grad()`, and what occurs to memory consumption and runtime if it is omitted?

Problem 5: VGG Factorization and Receptive Field Equivalence

VGG-16 replaced large convolution filters (such as 7×7 and 5×5 used in AlexNet) with a stack of small 3×3 filters.

1. Prove mathematically that a stack of two consecutive 3×3 convolution layers (with stride 1 and no pooling between them) has the exact same effective receptive field as a single 5×5 convolution layer.
2. Calculate the ratio of parameters between a stack of two 3×3 filters versus a single 5×5 filter (assuming C input and C output channels). Why is this factorization advantageous?

Problem 6: ResNet Residual Learning & Gradient Highway

1. Write down the residual mapping formulation $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$.
2. Differentiate the residual mapping with respect to input $\mathbf{x}$ to derive the gradient flow equation:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}} = \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right)$$

3. Explain mathematically why the presence of the identity term $\mathbf{I}$ prevents the vanishing gradient problem even when networks exceed 100 layers.

Problem 7: Stemming vs Lemmatization

Compare the theoretical foundations and outputs of Stemming (e.g., Porter Stemmer) and Lemmatization (e.g., WordNet Lemmatizer with POS tags):

1. Process the words `"running"`, `"better"`, `"flies"`, and `"corpora"` using both methods.
2. Explain two failure cases of stemming (over-stemming and under-stemming) with concrete examples.

Problem 8: One-Hot Sparsity & Dot-Product Blindness

Suppose a natural language vocabulary has $|\mathcal{V}| = 100,000$ unique words.

1. What is the memory requirement (in Megabytes) to store the one-hot vectors of a document containing 500 words using 32-bit floating point numbers?
2. Prove that the cosine similarity between the one-hot vector of `"king"` and `"queen"` is zero, and explain why this proves one-hot encoding fails to capture semantic meaning.

Problem 9: Bag-of-Words Document-Term Matrix Construction

Consider a toy corpus with 2 documents:

- D_1: `"deep learning models learn patterns"`
- D_2: `"deep neural models learn deep patterns"`

1. Extract the sorted alphabetical vocabulary $\mathcal{V}$.
2. Construct the exact $2 \times |\mathcal{V}|$ Document-Term Matrix (DTM).
3. If document $D_3 =$ "transformers generate text" is passed at test time, how does the standard DTM handle it?

Problem 10: N-Gram Negation Handling

Consider the two sentences:

- S_1: `"The film was not good, it was bad."`
- S_2: `"The film was not bad, it was good."`

1. Show why standard Bag-of-Words (Unigrams) fails to differentiate the sentiments of S_1 and S_2.
2. List the bigrams ($n = 2$) for both sentences and identify the critical bigrams that enable a linear model to distinguish them.

Problem 11: Step-by-Step TF-IDF Calculation

Consider a corpus of $N = 4$ documents:

- D_1: `"cat sat mat"`
- D_2: `"cat cat cat"`
- D_3: `"dog bark loud"`
- D_4: `"cat dog friends"`

Using standard formulas $\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}$ and $\text{IDF}(t) = \log_{10}\left(\frac{N}{\text{DF}(t)}\right)$:

1. Calculate $\text{TF}(\text{"cat"}, D_2)$ and $\text{TF}(\text{"cat"}, D_1)$.
2. Calculate $\text{IDF}(\text{"cat"})$ and $\text{IDF}(\text{"mat"})$.
3. Compute the unnormalized TF-IDF weight for `"cat"` in D_2 and `"mat"` in D_1.

Problem 12: Word2Vec: CBOW vs Skip-Gram Dynamics

1. Describe the architectural and mathematical difference in training objectives between Continuous Bag-of-Words (CBOW) and Skip-Gram.
2. Given embedding vectors $\mathbf{u} = [0.8, 0.6, 0.0]$ and $\mathbf{v} = [0.6, 0.8, 0.0]$, calculate their exact Cosine Similarity.
3. How does vector arithmetic allow Word2Vec to solve analogy questions like $\vec{v}_{\text{King}} - \vec{v}_{\text{Man}} + \vec{v}_{\text{Woman}} \approx \vec{v}_{\text{Queen}}$?

Problem 13: Recurrent Neural Network Parameter Count & Unrolling

A single-layer vanilla RNN has input size $d_x = 100$, hidden state size $d_h = 256$, and output size $d_y = 10$ (for multi-class classification).

1. State the exact shapes and calculate the number of parameters for:
    - $\mathbf{W}_{xh}$
    - $\mathbf{W}_{hh}$
    - $\mathbf{b}_h$
    - $\mathbf{W}_{hy}$
    - $\mathbf{b}_y$
2. Total number of parameters in the RNN cell.
3. Does this parameter count change if the input sequence length increases from $T = 10$ to $T = 1,000$? Explain why.

Problem 14: BPTT Gradient Vanishing Mechanics & Gradient Clipping

1. During Backpropagation Through Time (BPTT), the gradient of loss $\mathcal{L}_t$ with respect to hidden state $\mathbf{h}_k$ requires computing $\prod_{j=k+1}^{t} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$. Show that this product contains powers of the recurrent weight matrix $\mathbf{W}_{hh}^T$.
2. Why does the hyperbolic tangent activation $\tanh(z)$ exacerbate the vanishing gradient problem?
3. If an unclipped gradient vector has norm $\|\mathbf{g}\|_2 = 25.0$ and we apply gradient norm clipping with threshold $\theta = 5.0$, what is the exact scaled gradient vector $\mathbf{g}_{\text{clipped}}$?

Problem 15: LSTM Cell Gates & CEC Mathematical Proof

1. List all 4 core equations of an LSTM cell (Forget gate, Input gate & Candidate, Cell State update, Output gate & Hidden state update).
2. The cell state update is $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$. Compute the partial derivative $\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}}$.
3. Explain why this partial derivative enables the Constant Error Carousel (CEC) to transmit gradient signals across hundreds of time steps without exponential decay.